



UNIVERSIDADE ESTADUAL PAULISTA (UNESP)

Instituto de Ciência e Tecnologia de Sorocaba

Curso: Engenharia de Controle e Automação

Disciplina: Estrutura de Dados

THÉO CARMONA DE SOUZA

**SISTEMA DE MONITORAMENTO INDUSTRIAL DE SENSORES:
IMPLEMENTAÇÃO E AVALIAÇÃO DE ESTRUTURAS DE DADOS
EM C++17**

Professor: Leopoldo André Dutra Lusquino Filho

Sorocaba

2026

THÉO CARMONA DE SOUZA

**SISTEMA DE MONITORAMENTO INDUSTRIAL DE SENSORES:
IMPLEMENTAÇÃO E AVALIAÇÃO DE ESTRUTURAS DE DADOS
EM C++17**

Relatório técnico apresentado à disciplina de Estrutura de Dados como parte da avaliação do Projeto de Estrutura de Dados 2026.

Professor: Leopoldo André Dutra Lusquino Filho

Curso: Engenharia de Controle e Automação

Instituição: Universidade Estadual Paulista (UNESP) - Instituto de Ciência e Tecnologia de Sorocaba.

Sorocaba

2026

RESUMO

Este relatório apresenta o desenvolvimento de um sistema simulado de monitoramento industrial de sensores, implementado em C++17, com o objetivo de armazenar, consultar, remover, tratar e analisar leituras contínuas de sensores. O sistema utiliza um conjunto de dados sintético com 10.000 registros, contendo identificador do sensor, timestamp, temperatura, pressão, umidade e vibração. A proposta compara estruturas clássicas de dados, como lista encadeada, árvore AVL e tabela hash, com estruturas adicionais, como Skip List e Counting Bloom Filter. Também é considerada uma tabela hash otimizada, comparada com a versão original. Foram implementadas operações adicionais de estatística, histograma, detecção de anomalias, filtragem por intervalo e ordenação por campo. Para avaliar o comportamento das estruturas, foram executados benchmarks de inserção, busca, remoção, memória estimada, colisões, latência média e escalabilidade. Além disso, foram realizados testes restritivos cobrindo memória, processamento, latência, qualidade dos dados e restrição algorítmica. Os resultados mostram que estruturas baseadas em hash apresentaram melhor tempo de consulta no cenário avaliado, enquanto a lista encadeada apresentou maior degradação em buscas. A análise evidencia a importância de escolher estruturas de dados de acordo com a operação predominante e com as restrições do domínio.

Palavras-chave: estruturas de dados; C++17; sensores industriais; benchmark; AVL; tabela hash; Skip List; Bloom Filter.

ABSTRACT

This report presents the development of a simulated industrial sensor monitoring system implemented in C++17. The system stores, searches, removes, validates and analyzes continuous sensor readings. The synthetic dataset contains 10,000 records composed of sensor identifier, timestamp, temperature, pressure, humidity and vibration. The project compares classical data structures, such as linked list, AVL tree and hash table, with additional structures, such as Skip List and Counting Bloom Filter. An optimized hash table is also evaluated against the original hash table. Additional operations include statistical analysis, histogram generation, anomaly detection, timestamp filtering and field-based sorting. The experimental evaluation includes insertion, search and removal time, estimated memory usage, collision rate, average latency and scalability. Restrictive tests were also performed to simulate memory, processing, latency, data quality and algorithmic constraints. The results indicate that hash-based structures achieved the best query times in the evaluated scenario, while the linked list showed the strongest degradation during searches. The analysis highlights the importance of choosing data structures according to the dominant operation and the constraints of the application domain.

Keywords: data structures; C++17; industrial sensors; benchmark; AVL; hash table; Skip List; Bloom Filter.

SUMÁRIO

1 INTRODUÇÃO	6
2 OBJETIVOS	7
2.1 Objetivo geral	7
2.2 Objetivos específicos	7
3 DOMÍNIO ESCOLHIDO E DATASET	8
4 GERAÇÃO E TRATAMENTO DOS DADOS	9
5 ESTRUTURAS DE DADOS IMPLEMENTADAS	10
5.1 Lista encadeada	10
5.2 Árvore AVL	10
5.3 Tabela Hash	10
5.4 Skip List	10
5.5 Counting Bloom Filter	10
5.6 OptimizedHashTable	11
6 OPERAÇÕES ADICIONAIS DO SISTEMA	12
7 METODOLOGIA DE BENCHMARK	13
8 RESULTADOS DO BENCHMARK PRINCIPAL	14
9 ESCALABILIDADE	17
10 COMPARAÇÃO ENTRE HASHTABLE E OPTIMIZEDHASHTABLE	19
11 TESTES EM CONDIÇÕES RESTRITIVAS	20

12 ANÁLISE DE COMPLEXIDADE	21
13 VISUALIZAÇÃO E INTERAÇÃO	22
14 DISCUSSÃO CRÍTICA DOS RESULTADOS	23
15 CONCLUSÃO	24
REFERÊNCIAS	25

1 INTRODUÇÃO

O desenvolvimento de sistemas computacionais que operam sobre dados contínuos exige a escolha adequada de estruturas de dados. Em aplicações reais, como sistemas industriais, redes de sensores e sistemas embarcados, leituras são produzidas de forma constante e precisam ser armazenadas, consultadas, atualizadas, removidas e analisadas com eficiência. Nesse contexto, o projeto desenvolvido simula um ambiente de monitoramento industrial no qual diferentes estruturas de dados são avaliadas sob as mesmas operações básicas.

O sistema foi implementado em C++17 e organizado em módulos para geração de dados, tratamento, armazenamento, análise, visualização e benchmark. A chave principal utilizada para comparação entre as estruturas foi o campo timestamp, pois cada leitura gerada recebe um instante único. Essa decisão permite testar inserção, busca e remoção de maneira uniforme entre estruturas como AVL, HashTable, LinkedList, SkipList e OptimizedHashTable.

O trabalho também considera que nem todas as estruturas possuem a mesma finalidade. Por esse motivo, o Counting Bloom Filter foi analisado separadamente como estrutura probabilística de pertinência, e não como uma busca exata equivalente às demais. Essa separação evita uma comparação metodologicamente inadequada e melhora a interpretação dos resultados.

2 OBJETIVOS

2.1 Objetivo geral

O objetivo geral do projeto é implementar e avaliar um sistema computacional capaz de gerenciar leituras de sensores industriais, comparando o comportamento de diferentes estruturas de dados em operações de inserção, busca, remoção, análise estatística, visualização e execução sob restrições.

2.2 Objetivos específicos

- Implementar operações básicas de inserção, busca e remoção em diferentes estruturas de dados.
- Utilizar pelo menos três estruturas tradicionais e duas estruturas adicionais fora da ementa básica.
- Gerar e tratar um dataset sintético com volume suficiente para comparação experimental.
- Executar operações adicionais de estatística, anomalias, filtragem e ordenação.
- Comparar uma estrutura otimizada com sua versão original.
- Medir tempo de execução, memória estimada, colisões, latência e escalabilidade.
- Executar cinco testes restritivos cobrindo memória, processamento, latência, dados e restrição algorítmica.
- Gerar visualizações no terminal e arquivos CSV para apoiar a análise.

3 DOMÍNIO ESCOLHIDO E DATASET

O domínio escolhido foi o monitoramento industrial de sensores. Essa escolha é adequada porque sensores industriais produzem dados contínuos, apresentam variações físicas reais, podem sofrer ruídos ou falhas e exigem consulta rápida a registros históricos. Um sistema desse tipo precisa lidar com grande quantidade de registros e com operações frequentes de busca, remoção e análise.

Cada registro de sensor é representado por seis campos: idSensor, timestamp, temperatura, pressão, umidade e vibração. O idSensor identifica o equipamento responsável pela leitura. O timestamp representa o instante lógico da medição e foi usado como chave principal. Os demais campos representam variáveis físicas monitoradas em ambiente industrial.

O dataset utilizado é sintético e possui 10.000 registros válidos. A geração sintética foi escolhida por permitir controle sobre a distribuição dos dados, reprodutibilidade dos experimentos e simulação de cenários restritivos, como dados corrompidos, anomalias e restrições de latência. A quantidade de 10.000 registros atende ao volume mínimo solicitado e permite observar diferenças práticas entre estruturas com custos assintóticos distintos.

4 GERAÇÃO E TRATAMENTO DOS DADOS

A geração dos dados foi realizada por meio do módulo Generator, com semente fixa igual a 42. O uso de seed fixa torna os experimentos reproduzíveis, permitindo que diferentes execuções produzam o mesmo conjunto de leituras e, consequentemente, possam ser comparadas com maior controle.

Após a geração, cada leitura passa pelo módulo DataTreatment. Esse módulo realiza normalização e validação dos registros. A normalização busca reduzir discrepâncias de escala entre campos físicos diferentes, enquanto a validação impede que registros inválidos sejam inseridos nas estruturas. Essa etapa é importante porque dados inválidos poderiam distorcer estatísticas, alterar resultados de busca e comprometer os benchmarks.

O fluxo adotado foi: gerar a leitura, atribuir um timestamp único, normalizar o registro, validar seus campos e inserir apenas os dados considerados válidos nas estruturas. Os registros descartados são contabilizados, de modo que o usuário possa visualizar a qualidade do dataset carregado.

5 ESTRUTURAS DE DADOS IMPLEMENTADAS

5.1 Lista encadeada

A lista encadeada foi usada como estrutura tradicional e como referência para demonstrar o custo de buscas lineares. Sua vantagem está na simplicidade de implementação e no baixo custo de inserção quando a operação é feita em posição controlada. Sua limitação principal é a busca por chave, que exige percorrer os nós sequencialmente, resultando em custo $O(n)$.

5.2 Árvore AVL

A árvore AVL representa uma árvore binária de busca balanceada. Sua principal vantagem é manter a altura controlada por rotações, garantindo busca, inserção e remoção em $O(\log n)$. No projeto, ela é adequada para consulta por timestamp quando se deseja desempenho previsível e ordenação implícita dos registros.

5.3 Tabela Hash

A HashTable foi implementada como estrutura tradicional para acesso rápido por chave. Em média, operações de inserção, busca e remoção tendem a $O(1)$, desde que a função hash distribua bem as chaves e a taxa de colisões seja controlada. Sua principal limitação é depender da estratégia de tratamento de colisões e da capacidade da tabela.

5.4 Skip List

A SkipList é uma estrutura probabilística formada por níveis de atalhos sobre uma lista ordenada. Seu comportamento esperado para inserção, busca e remoção é $O(\log n)$. Ela foi usada como alternativa fora da ementa, permitindo comparar uma abordagem probabilística com estruturas balanceadas como a AVL.

5.5 Counting Bloom Filter

O Bloom Filter é uma estrutura probabilística usada para testes de pertinência. Como o Bloom Filter clássico não permite remoção segura, o projeto utiliza uma variação com contadores inteiros, chamada Counting Bloom Filter. Essa modificação permite incrementar contadores na inserção e decrementá-los na remoção. A desvantagem é o maior consumo de memória em relação a um Bloom Filter baseado apenas em bits.

Por ser probabilístico, o Bloom Filter pode indicar falsos positivos, isto é, responder que um elemento possivelmente está presente mesmo quando não está. Por isso, seus resultados não foram comparados diretamente com estruturas de busca exata.

5.6 OptimizedHashTable

A OptimizedHashTable foi usada como estrutura otimizada do projeto. Ela foi comparada com a HashTable original, considerando tempo de inserção, busca, remoção, colisões, taxa de colisão, uso estimado de memória e latência média. Essa comparação atende ao requisito de escolher pelo menos uma estrutura e otimizá-la para o contexto do dataset.

6 OPERAÇÕES ADICIONAIS DO SISTEMA

Além das operações básicas de inserção, busca e remoção, o sistema implementa operações adicionais de análise e exploração dos dados. As principais são estatísticas completas, histograma de temperatura, detecção de anomalias, filtragem por intervalo de timestamp e ordenação por campo.

As estatísticas calculam média, mínimo, máximo e desvio padrão para temperatura, pressão, umidade e vibração. O histograma permite visualizar a distribuição de temperatura em faixas. A detecção de anomalias identifica leituras fora de padrões esperados. A filtragem por timestamp permite recuperar subconjuntos temporais, e a ordenação por campo permite examinar extremos de temperatura, pressão, umidade, vibração ou timestamp.

7 METODOLOGIA DE BENCHMARK

Os benchmarks foram planejados para medir diferentes dimensões de desempenho: tempo de inserção, tempo total de busca, tempo médio de consulta, tempo de remoção, latência média combinada, uso estimado de memória e colisões. As estruturas de busca exata foram avaliadas em grupo separado do Bloom Filter, porque este último é probabilístico e não oferece a mesma semântica de consulta.

A memória foi estimada com base no tamanho dos nós e estruturas internas, usando sizeof. Essa estimativa não representa com exatidão o overhead do alocador do sistema operacional, mas permite comparação aproximada entre estruturas executadas no mesmo ambiente.

As medições foram exportadas em arquivos CSV e depois transformadas em tabelas e gráficos. Essa estratégia permite reproduzir os resultados, gerar visualizações externas e documentar os trade-offs entre tempo de execução e consumo de memória.

8 RESULTADOS DO BENCHMARK PRINCIPAL

Tabela 1 - Resultados do benchmark principal

Estrutura	Inserção (ms)	Busca (ms)	Busca média (μs)	Remoção (ms)	Latência (μs)	Memória (KB)	Colisões	Taxa (%)
AVL	7,534	0,190	0,190	0,763	0,707	492,188	N/A	N/A
Hash Table	0,544	0,039	0,039	0,066	0,054	507,898	0	0,000
Lista Encadeada	1,179	16,548	16,548	21,677	3,284	351,562	N/A	N/A
SkipList	12,527	0,648	0,648	1,019	1,183	914,062	N/A	N/A
Hash Otimizada	2,363	0,071	0,071	0,110	0,212	460,164	4213	46,811
Bloom Filter	1,580	0,154	0,154	0,274	0,167	390,637	6964	19,344

Fonte: resultados gerados pelo sistema implementado.

A Tabela 1 mostra que a HashTable apresentou o menor tempo de busca no cenário avaliado, seguida pela OptimizedHashTable e pela AVL. A LinkedList apresentou tempo de busca muito superior, o que confirma a limitação esperada de uma estrutura linear. A SkipList teve desempenho intermediário, mas com custo de inserção mais elevado no experimento, possivelmente devido ao gerenciamento de níveis probabilísticos.

Em termos de memória estimada, a lista encadeada e o Bloom Filter apresentaram valores relativamente baixos, enquanto a SkipList consumiu mais memória por manter múltiplos níveis de ponteiros. A OptimizedHashTable consumiu menos memória estimada do que a HashTable original no experimento, embora tenha registrado colisões no contador da implementação. Isso indica que a otimização não deve ser avaliada apenas por colisões, mas pelo conjunto de métricas: memória, tempo e latência.

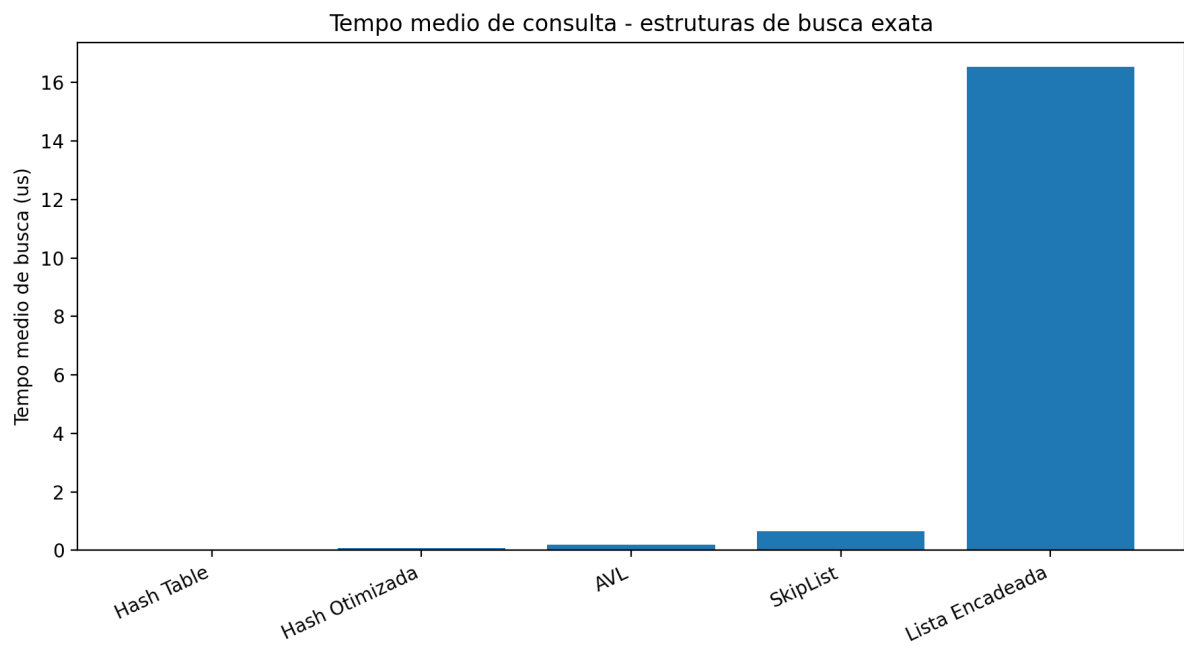


Figura 1 - Tempo médio de consulta por estrutura.

Fonte: resultados gerados pelo sistema implementado.

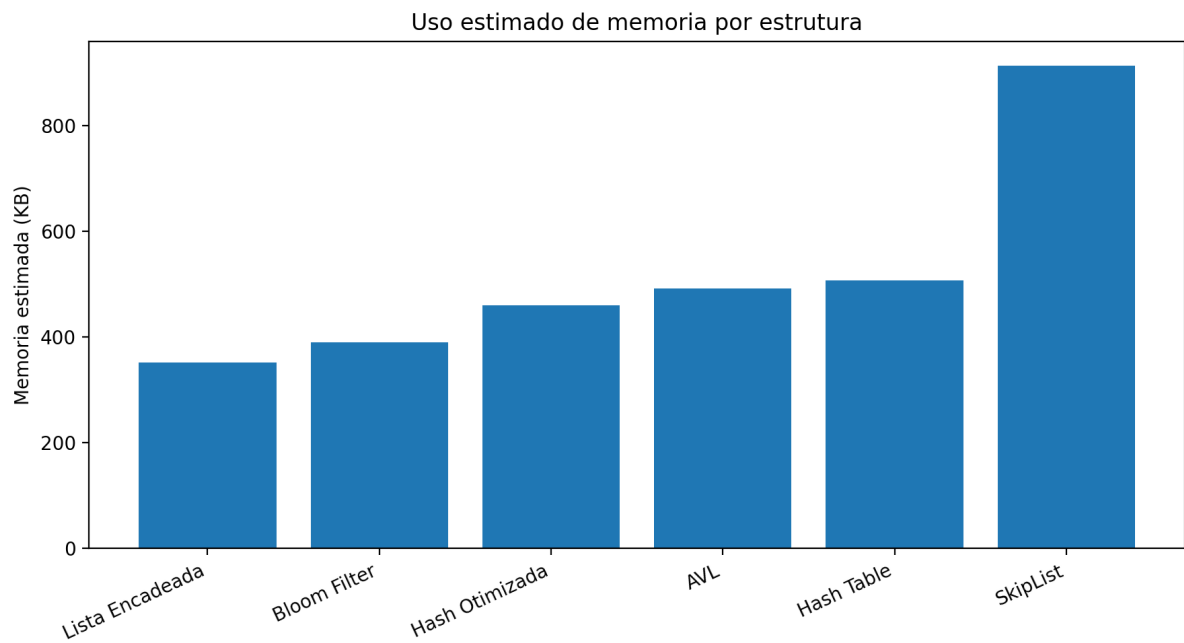


Figura 2 - Uso estimado de memória por estrutura.

Fonte: resultados gerados pelo sistema implementado.

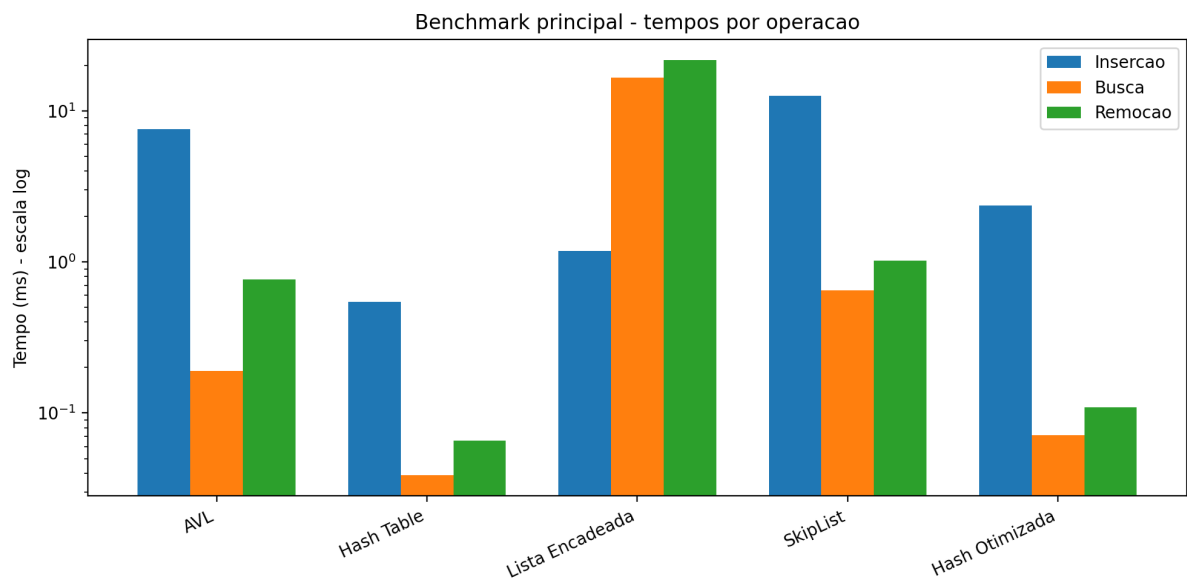


Figura 3 - Comparação de tempos de inserção, busca e remoção.

Fonte: resultados gerados pelo sistema implementado.

9 ESCALABILIDADE

O teste de escalabilidade avaliou o comportamento das estruturas com 1.000, 5.000, 10.000 e 50.000 registros. Essa análise permite observar se o crescimento prático acompanha a complexidade teórica esperada.

Tabela 2 - Resultados do benchmark de escalabilidade

Tamanho	AVL (ms)	Hash (ms)	Lista (ms)	SkipList (ms)	Hash otimizada (ms)
1000	0,594	0,167	2,968	0,540	0,385
5000	3,043	1,106	10,332	3,615	1,526
10000	6,470	1,129	17,680	14,240	3,192
50000	47,003	5,896	23,021	415,313	20,913

Fonte: resultados gerados pelo sistema implementado.

A HashTable manteve crescimento reduzido ao aumentar o volume de dados, o que é coerente com o comportamento médio esperado de estruturas hash. A AVL apresentou crescimento maior, mas ainda compatível com uma estrutura logarítmica. A SkipList apresentou aumento expressivo no cenário de 50.000 registros, demonstrando que a implementação e os fatores constantes podem influenciar fortemente o desempenho observado.

A LinkedList, por sua vez, é prejudicada principalmente por buscas lineares. Mesmo quando o crescimento total do tempo não aparece como perfeitamente quadrático, a tendência de degradação em operações de busca é clara quando comparada às estruturas de acesso mais eficiente.

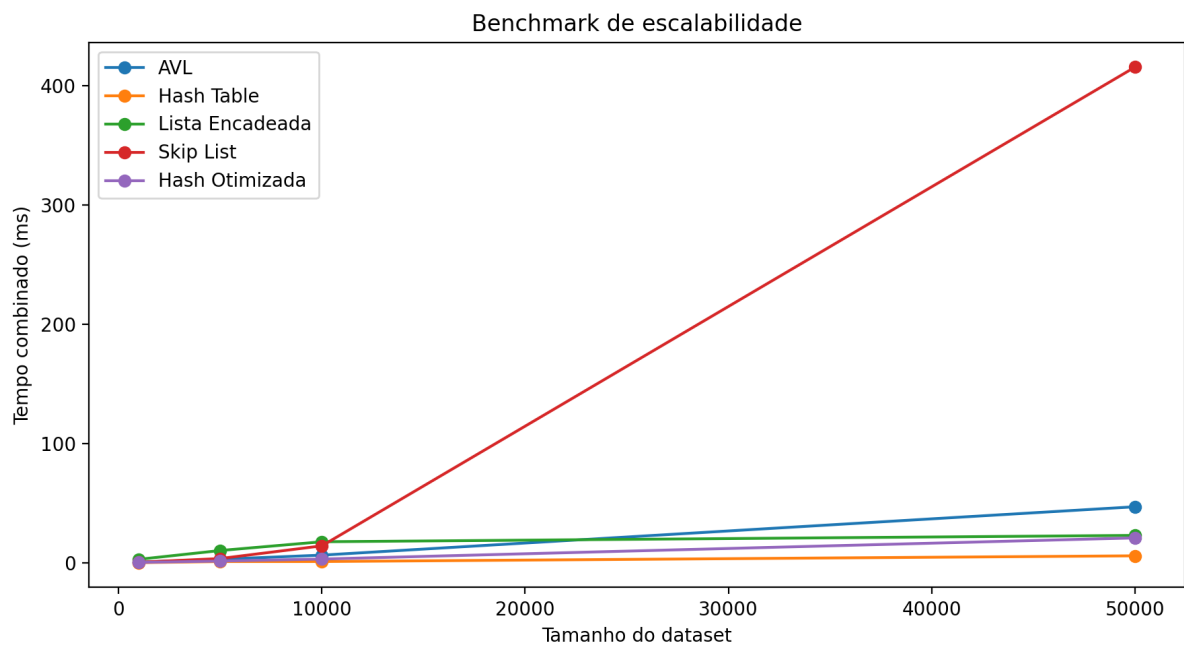


Figura 4 - Benchmark de escalabilidade com tamanhos crescentes.

Fonte: resultados gerados pelo sistema implementado.

10 COMPARAÇÃO ENTRE HASHTABLE E OPTIMIZEDHASHTABLE

A comparação entre HashTable e OptimizedHashTable é relevante porque o projeto exige a otimização de pelo menos uma estrutura e sua comparação com a versão original. A HashTable original apresentou busca extremamente rápida e não registrou colisões no contador da execução tratada. A OptimizedHashTable teve busca rápida, menor memória estimada e latência competitiva, mas registrou colisões no experimento.

Esse resultado demonstra que uma otimização deve ser analisada por múltiplos critérios. A versão otimizada não foi superior em todos os indicadores, mas apresentou comportamento competitivo e redução de memória estimada. Em cenários reais, a escolha entre as duas dependeria do peso dado a memória, previsibilidade, distribuição das chaves e estratégia de colisão.

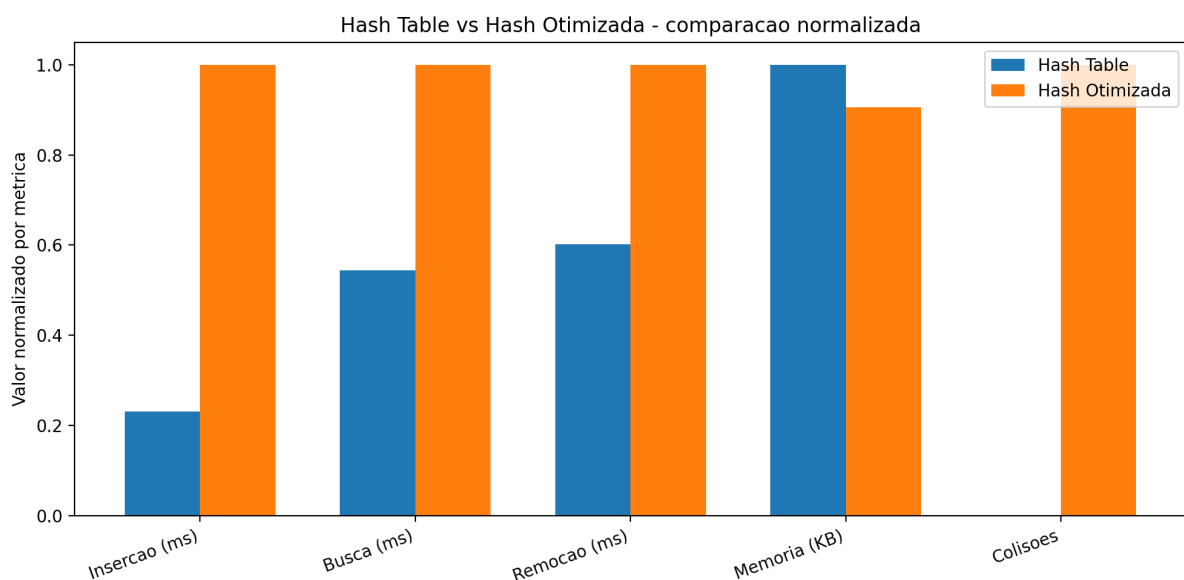


Figura 5 - Comparação entre HashTable original e HashTable otimizada.

Fonte: resultados gerados pelo sistema implementado.

11 TESTES EM CONDIÇÕES RESTRITIVAS

Foram executados cinco testes restritivos, cada um associado a uma categoria diferente: memória, processamento, latência, dados e restrição algorítmica. Essa divisão permite avaliar o comportamento das estruturas em condições mais próximas de sistemas reais, nos quais nem sempre há recursos ideais ou dados perfeitamente confiáveis.

Tabela 3 - Resultados dos testes restritivos

Teste	Resultado observado	Categoria interpretada
R2	9500 rejeicoes	Memória - limite de capacidade da AVL
R7	buscas avaliadas	Processamento - orçamento por busca
R12	20193.268896x	Latência - atraso artificial em consultas
R16	2000 rejeitados	Dados - leituras corrompidas
R21	169.512544x	Algorítmica - substituição por estrutura menos eficiente

Fonte: resultados gerados pelo sistema implementado.

No teste R2, a AVL limitada a 500 elementos rejeitou 9.500 inserções ao tentar receber 10.000 registros. Esse resultado simula uma restrição de memória e mostra que o sistema precisa lidar com a impossibilidade de armazenar todos os dados quando a capacidade é limitada.

No teste R7, as buscas foram avaliadas contra um orçamento máximo por operação. Esse cenário simula sistemas em tempo real ou quase tempo real, nos quais uma operação não pode ultrapassar certo limite de tempo. Estruturas lineares tendem a ser mais vulneráveis a esse tipo de restrição.

No teste R12, a inserção de atraso artificial causou aumento expressivo no tempo observado, demonstrando que latência externa pode dominar o custo total mesmo quando a estrutura de dados é eficiente. Esse resultado é importante porque, em sistemas reais, o desempenho não depende apenas do algoritmo, mas também de comunicação, disco, rede e outros componentes.

No teste R16, a corrupção de 20% dos dados resultou em 2.000 registros rejeitados pelo tratamento. Isso evidencia a importância da validação antes da inserção nas estruturas. Já o teste R21 mostrou que substituir AVL por LinkedList degradou a busca em aproximadamente 169,5 vezes, confirmando a diferença prática entre $O(\log n)$ e $O(n)$ em operações por chave.

12 ANÁLISE DE COMPLEXIDADE

Tabela 4 - Complexidade teórica das estruturas

Estrutura	Inserção	Busca	Remoção	Observação
LinkedList	$O(1)$ ou $O(n)$	$O(n)$	$O(n)$	Busca linear por timestamp.
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balanceamento por rotações.
HashTable	$O(1)$ médio	$O(1)$ médio	$O(1)$ médio	Depende da distribuição da função hash.
SkipList	$O(\log n)$ esperado	$O(\log n)$ esperado	$O(\log n)$ esperado	Estrutura probabilística com níveis.
Counting Bloom Filter	$O(k)$	$O(k)$	$O(k)$	Consulta probabilística de pertinência.
OptimizedHashTable	$O(1)$ médio	$O(1)$ médio	$O(1)$ médio	Versão otimizada para comparação.

Fonte: elaboração própria com base nas estruturas implementadas.

A comparação entre teoria e prática mostra que a complexidade assintótica é essencial para prever tendências, mas não explica sozinha todos os resultados. Fatores como alocação dinâmica, cache, constantes de implementação, função hash, número de ponteiros e geração probabilística de níveis afetam os tempos medidos. Por isso, a análise experimental complementa a análise teórica.

13 VISUALIZAÇÃO E INTERAÇÃO

A visualização do sistema foi implementada via terminal. O programa apresenta tabelas de benchmark, estatísticas dos sensores, histograma de temperatura, comparação entre estruturas e menu interativo. O modo automático executa todos os testes sem entrada do usuário, permitindo compatibilidade com GitHub Actions. O modo interativo permite inserir, buscar, remover, filtrar, ordenar, visualizar estatísticas e executar benchmarks sob demanda.

Além da visualização em terminal, os resultados são exportados em CSV nos arquivos `output/benchmark.csv`, `output/escalabilidade.csv` e `output/restricoes.csv`. Esses arquivos foram utilizados para gerar gráficos e tabelas apresentados neste relatório.

14 DISCUSSÃO CRÍTICA DOS RESULTADOS

Os resultados confirmam que não existe uma única estrutura ideal para todos os cenários. A HashTable foi a mais eficiente em busca no conjunto avaliado, mas depende de boa distribuição de chaves e controle de colisões. A AVL apresentou desempenho estável e previsível, sendo adequada quando se deseja garantia logarítmica. A LinkedList, embora simples, mostrou limitação clara em buscas por timestamp.

A SkipList apresentou comportamento esperado em parte dos testes, mas seu custo aumentou significativamente em escala maior. Isso pode ser explicado por fatores de implementação, número de níveis, uso de ponteiros e custos constantes. Esse caso mostra que uma estrutura com boa complexidade esperada pode ter desempenho prático inferior dependendo da implementação e do cenário de uso.

A análise do Bloom Filter exige cuidado metodológico. Como ele não realiza busca exata, não seria correto compará-lo diretamente com AVL, HashTable ou SkipList como se todas respondessem à mesma pergunta. Por esse motivo, o projeto separou o Bloom Filter em um benchmark probabilístico. Essa decisão melhora a validade da análise e evita conclusões equivocadas.

Os testes restritivos mostraram que o desempenho do sistema também é afetado por fatores externos à estrutura escolhida. A latência artificial, por exemplo, aumentou fortemente o tempo total, mesmo que a operação interna de busca seja rápida. Já a corrupção de dados reforçou a importância do tratamento antes do armazenamento.

15 CONCLUSÃO

O sistema desenvolvido cumpriu o objetivo de implementar e comparar diferentes estruturas de dados em um domínio simulado de monitoramento industrial de sensores. A implementação contemplou estruturas tradicionais, estruturas adicionais, uma estrutura otimizada, operações básicas, operações analíticas, benchmark, testes restritivos, visualização e exportação de resultados.

Os resultados reforçam que a escolha da estrutura de dados depende do problema. Estruturas hash são adequadas para consultas rápidas por chave, árvores balanceadas fornecem desempenho previsível, listas encadeadas são simples mas limitadas para busca, Skip Lists oferecem alternativa probabilística e Bloom Filters são úteis para pertinência probabilística com consumo controlado.

A principal conclusão é que o domínio de aplicação, a operação predominante e as restrições do ambiente devem orientar a escolha da estrutura. A análise experimental mostrou diferenças relevantes de tempo, memória, escalabilidade e comportamento sob restrições, demonstrando a importância de combinar teoria e prática na avaliação de estruturas de dados.

REFERÊNCIAS

- BLOOM, Burton H. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, v. 13, n. 7, p. 422-426, 1970.
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- KNUTH, Donald E. *The Art of Computer Programming: Sorting and Searching*. 2. ed. Boston: Addison-Wesley, 1998. v. 3.
- PUGH, William. Skip lists: a probabilistic alternative to balanced trees. *Communications of the ACM*, v. 33, n. 6, p. 668-676, 1990.
- SEEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- SOUZA, Théo. *Sistema de Monitoramento Industrial de Sensores: repositório do projeto Estrutura de Dados 2026*. GitHub, 2026. Disponível em: <https://github.com/Homanders/Estrutura-de-dados>. Acesso em: 29 jun. 2026.
- UNIVERSIDADE ESTADUAL PAULISTA. *Projeto de Estrutura de Dados 2026*. Enunciado da disciplina de Estrutura de Dados, 2026.